

8 Laboratorinis darbas. Difi-Helmano apsiskeitimo slaptaisiais raktais algoritmas

Saugus rakto apsiskeitimas

Simetrinės šifravimo kriptosistemos yra daug greitesnės už asimetrines, todėl kai mums svarbi greitimeika, geriau naudoti simetrines kriptosistemas. Tokiose sistemose, siuntėjas ir gavėjas turi naudoti tą patį raktą. Kaip jiems sutarti dėl rakto taip, kad jo niekas nesužinotų? Šią problemą išsprendžia Difi-Helmano (DH) rakto apsiskeitimo algoritmas.

Generuojantys elementai

Skaičius g yra generuojantis pirminiu moduliu p , kai visos $g^a \bmod p$ liekanos yra unikalios, čia $0 < a < p$, $a \in \mathbb{Z}$.

a	$2^a \bmod 11$	a	$2^a \bmod 11$
1	2	6	9
2	4	7	7
3	8	8	3
4	5	9	6
5	10	10	1

a	$3^a \bmod 11$	a	$3^a \bmod 11$
1	3	6	3
2	9	7	9
3	5	8	5
4	4	9	4
5	1	10	1

Kai $p = 11$, $g = 2$ yra generuojantis elementas, o $g = 3$ - ne.

Generuojantys elementai pirminiu moduliu leidžia konvertuoti vieną sveikąjį skaičių į kitą. Žinant a apskaičiuoti $g^a \bmod p$ yra lengva, o žinant $g^a \bmod p$ apskaičiuoti a tektų išspręsti diskretųjį logaritmą.

DH algoritmo saugumas yra paremtas tuo, kad labai sunku apskaičiuoti diskretųjį logaritmą $g^a \bmod p$.

Tarkime, kad klientas ir serveris nori apsieisti simetrinės kriptosistemos raktais. DH atrodytų taip:

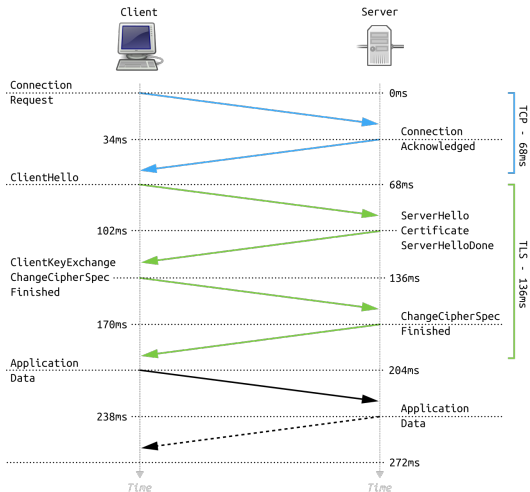
- 1 Klientas ir serveris viešai sutaria dėl parametrų p ir g . Čia p - pirminis skaičius, g - generuojantis elementas moduliu p . Šiuo atveju juos pasiūlys serveris.
- 2 Klientas sugalvoja slaptą skaičių a , $0 < a < p$ ir apskaičiuoja $A \equiv g^a \pmod{p}$. Šią reikšmę nusiunčia serveriui.
- 3 Serveris sugalvoja slaptą skaičių b , $0 < b < p$ ir apskaičiuoja $B \equiv g^b \pmod{p}$. Šią reikšmę nusiunčia klientui.
- 4 Klientas apskaičiuoja $s \equiv B^a \pmod{p}$.
- 5 Serveris apskaičiuoja $s \equiv A^b \pmod{p}$.
- 6 Abi pusės sutarė dėl slapto rakto. Algoritmas baigtas.

Parametrų slaptumas

Viešai paskelbiamos reikšmės p , g , A ir B , bet nežinant a arba b rasti s labai sudėtinga.

- 1 Tarkime serveris nustatė, kad $p = 23$, o $g = 5$. Šias reikšmes nusiunčia klientui.
- 2 Klientas parenka $a = 6$ ir apskaičiuoja $A = g^a \bmod p = 5^6 \bmod 23 = 8$. Gautą reikšmę nusiunčia serveriui.
- 3 Serveris parenka $b = 15$ ir apskaičiuoja $B = g^b \bmod p = 5^{15} \bmod 23 = 19$. Gautą reikšmę nusiunčia klientui.
- 4 Klientas apskaičiuoja $s = B^a \bmod p = 19^6 \bmod 23 = 2$.
- 5 Serveris apskaičiuoja $s = A^b \bmod p = 8^{15} \bmod 23 = 2$

TLS handshake



Apsikeitimas raktais naudojant RSA

TLS atveju, slaptais raktais apsieikti labai paprasta naudojant serverio viešąjį RSA raktą. Procesas atrodytų taip:

- 1 Klientas patikrina serverio sertifikatą.
- 2 Klientas sugalvoja slaptą raktą.
- 3 Klientas šį raktą užšifruoja serverio viešuoju RSA raktu ir nusiunčia jį serveriui.
- 4 Klientas ir serveris turi tą patį raktą ir toliau bendravimą tęsia simetrinio rakto kriptosistema.

Šis metodas veikia, bet turi pažeidžiamumą. Jei, kadanors ateityje bus paviestas serverio privatusis RSA raktas, bus galima atšifruoti raktą kurį sugalvojo klientas. Jei simetrinės kriptosistemos raktas bus žinomas, bus galima atšifruoti visą susirašinėjimą tarp kliento ir serverio. Tai reiškia, kad toks rakto apsieitimo būdas negali užtikrinti slaptumo ateityje (angl. *forward secrecy*). Slaptumas ateityje garantuoja, kad sesijų raktai nebus atskleisti, net jei bus atskleistos ilgalaikės paslaptys.

Tinkamai implementuotas DH *forward secrecy* užtikrina.

DH raktai (reikšmės ir a ir b) gali būti statiški, efemeriški (trumpalaikiai, pergeneruojami kiekvienoje sesijoje) arba maišyti. Skirtingos raktų kombinacijos leidžia DH naudoti skirtingoms reikmėms.

Kai a ir b - statiški

Sugeneruoja ilgalaikę paslaptį. Neužtikrina *forward secrecy*.

Kai a ir b - efemeriški

Sugeneruoja trumpalaikę paslaptį. Užtikrina *forward secrecy*, bet nepatvirtina tapatybės. Naudojamas TLS. Dar vadinamas efemerišku DH arba EDH arba DHE.

Kai a - efemeriškas, b - statiškas

Naudojama kai kuriose asimetrinio rakto kriptosistemose pvz. ElGamalio. Kai naudojama raktui sutarti efemerinė pusė gali patvirtinti statinės pusės tapatybę. Neužtikrina *forward secrecy*.

DHE-RSA

TLS implementuoja tiek DH tiek DHE. Dėl *forward secrecy* trūkumo DH naudojamas retai, o nuo TLS 1.3 versijos nebepalaikomas. Autentiškumui patvirtinti vien DHE nepakanka, todėl kombinuojama su asimetrinėmis kriptosistemomis pvz. DHE-RSA.

DHE-RSA

Serveris nustato g ir p , šios reikšmės naudojamos su visais klientais.

- 1 Serveris sugeneruoja atsitiktinį b . Klientui nusiunčia sertifikatą, g , p , $B \equiv g^b \pmod{p}$ ir juos pasirašo privačiuoju RSA raktu.
- 2 Klientas patikrina serverio sertifikatą ir ar g , p , B buvo pasirašyti privačiuoju raktu.
- 3 Jei bent viena sąlyga nepatenkinta ryšys laikomas nesaugiu.
- 4 Klientas sugeneruoja atsitiktinį a ir apskaičiuoja $s \equiv B^a \pmod{p}$. Serveriui nusiunčia $A \equiv g^a \pmod{p}$.
- 5 Serveris apskaičiuoja $s \equiv A^b \pmod{p}$.

Man-in-the-middle ataka

Tarkime vartotojas nori apsiukeisti paslaptimi su serveriu, o užpuolikas yra nustatęs, kad pokalbis vyktų per jį. Tada užpuolikas gali sudaryti paslaptį s_1 su serveriu ir paslaptį s_2 su klientu. Tada užpuolikas pranešimus, kuriuos siunčia klientas, atšifruoja paslaptimi s_2 , perskaito ir užšifruoja paslaptimi s_1 išsiunčia serveriui. Tiek serveris, tiek klientas sėkmingai sudarė paslaptis DH, bet nežino su kuo.

Nuo šios atakos apsiginti paprasta, jei serverio siunčiami parametrai yra pasirašomi.

Diskretaus logaritmo sprendimas

Kai p pakankamai mažas diskretųjį logaritmą galima išspręsti. Vienas būdas atspėti a žinant $g^a \bmod p$. Toks metodas yra labai lėtas. Egzistuoja algoritmai leidžiantys pagreitinti šią procedūrą. pvz. D. Šenkso (*D. Shanks*) mažo žingsnio didelio žingsnio (angl. *baby-step giant-step*) algoritmas.

Algoritmas

Tarkime, turime g - generuojantį elementą, p - pirminį skaičių ir $x = g^a \bmod p$. Norime rasti a .

- 1 Apskaičiuojama $m = \lceil \sqrt{p-1} \rceil$

Baby-step:

- 2 Sukonstruojama raktų ir reikšmių aibė $r_1 = \{(j, g^{mj} \bmod p) : 0 \leq j < m\}$
- 3 Sukonstruojama raktų ir reikšmių aibė $r_2 = \{(i, x(g^{-i} \bmod p) \bmod p) : 0 \leq i < m\}$

Giant-step:

- 4 Aibėse r_1 ir r_2 randamos pasikartojančios reikšmės ir išsaugomi jų raktai i ir j
- 5 Grąžinama $a = (mj + i) \bmod (p-1)$

Tarkime $g = 3$, $p = 43$, $x = 3^a \bmod 43 = 35$. Ieškome a .

- ➊ Apskaičiuojama $m = \lceil \sqrt{43 - 1} \rceil = 7$
- ➋ Sukonstruojama aibė $r_1 = \{(j, 3^{7j} \bmod 43) : 0 \leq j < 7\}$

j	0	1	2	3	4	5	6
$g^{mj} \bmod p$	1	37	36	42	6	7	1

- ➌ Sukonstruojama aibė $r_2 = \{(i, 35(3^{-i} \bmod 43) \bmod 43) : 0 \leq i < 7\}$

i	0	1	2	3	4	5	6
$x(g^{-i} \bmod p) \bmod p$	35	26	23	22	36	12	4

- ➍ Abejose aibėse galima rasti 36. Todėl $j = 2$, $i = 4$,
 $a = (mj + i) \bmod (p - 1) = (7 \times 2 + 4) \bmod 42 = 18$.

- DH algoritmui reikalinga parinkti pirminį p ir generuojantį elementą g .
- Yra du būdai parinkti šias reikšmes: parinkti p ir rasti jo generuojančius elementus arba parinkti g ir rasti tokį p , kad g būtų generuojantis elementas.
- Didelis g saugumo nesuteikia, o mažas g leidžia pagreitinti skaičiavimus, todėl praktikoje yra naudojamas antras parametrų parinkimo variantas ir $g = 2$.
- Tada parenkamas atsitiktinis p ir tikrinama ar g yra generuojantis. Jei taip, parametrų parinkimo procedūra stabdoma, jei ne - renkamas naujas p .
- Naivus būdas patikrinti ar g generuojantis apskaičiuoti visas liekanas $g^a \bmod p$, kur $0 < a < p - 1$. Jei visos liekanos yra unikalios p tinka, jei yra pasikartojančių liekanų p netinka. Dideliems p toks metodas užtruktų per ilgai.

Tarkime, norime patikrinti, ar g yra pirminio p generuojantis elementas. Šį testą galima atlikti tokia procedūra:

- 1 Apskaičiuojama Eulerio funkcija $n = \phi(p) = p - 1$
- 2 Sudaroma n pirminių dauginamųjų aibė $f = \{f_1, f_2, \dots, f_k\}$.
- 3 Sudaroma n liekanų aibė $a = \{g^{\frac{n}{f_i}} \bmod p : 1 \leq i \leq k\}$.
- 4 Jei nors vienas aibės a elementas $a_i = 1$ - g nėra p generatorius. Jei ne, g yra p generatorius.

Tarkime, norime patikrinti, ar 2 yra pirminio 43 generuojantis elementas.

- 1 Apskaičiuojama Eulerio funkcija $n = \phi(43) = 43 - 1 = 42$
- 2 Sudaroma n pirminių dauginamųjų aibė $f = \{2, 3, 7\}$.
- 3 Sudaroma n liekanų aibė $a = \{$
 $2^{\frac{42}{2}} \bmod 43 = 2^{21} \bmod 43 \equiv 42$
 $2^{\frac{42}{3}} \bmod 43 = 2^{14} \bmod 43 \equiv 1$
 $2^{\frac{42}{7}} \bmod 43 = 2^6 \bmod 43 \equiv 21 \}$
- 4 $2^{14} \bmod 43$ liekana lygi 1, todėl 2 nėra generuojantis moduliui 43.

Tarkime, norime patikrinti, ar 3 yra pirminio 43 generuojantis elementas.

- 1 Pirmi 2 žingsniai tie patys. Sudaroma n liekanų aibė $a = \{$
 $3^{\frac{42}{2}} \bmod 43 = 3^{21} \bmod 43 \equiv 42$
 $3^{\frac{42}{3}} \bmod 43 = 3^{14} \bmod 43 \equiv 36$
 $3^{\frac{42}{7}} \bmod 43 = 3^6 \bmod 43 \equiv 41 \}$
- 2 Nei viena liekana nelygi 1, todėl 3 yra generuojantis moduliui 43.